

I want you to develop a complete Android mobile application called:

“Location-Based Block Silent Assistant”

Project type: Android Mobile Application Development Project

Main purpose: The application automatically changes the user's smartphone sound mode when the user enters or leaves predefined college locations such as classrooms, blocks, laboratories, library, seminar hall, auditorium, and examination hall.

The application should use Android location/geofencing functionality to detect predefined areas and automatically apply the sound-mode rule assigned to that location.

IMPORTANT: This must be a REAL, FUNCTIONAL Android application, not only a UI prototype.

Develop the complete project including:

- Android frontend
- Location/geofencing functionality
- Sound mode automation
- Local database/storage
- Notifications
- Timetable
- Activity history
- Settings
- Emergency override
- Permissions
- Error handling
- Battery optimization
- Privacy controls
- Testing

===== 1. TECHNOLOGY REQUIREMENTS  
=====

Develop this as a native Android application.

Preferred development environment:

- Android Studio
- Java
- Gradle
- Android SDK
- XML layouts for UI
- Android native APIs

Use Java rather than Kotlin.

Use Android-compatible libraries only.

Use:

- Google Maps SDK where required
- Android Location APIs
- Geofencing APIs
- Notification APIs
- Room Database or SQLite for local storage
- Android Runtime Permissions

The application should work without requiring a separate web server.

The application should store user settings, blocks, rules, timetable and activity history locally.

===== 2. APPLICATION NAME  
=====

App name:

Location-Based Block Silent Assistant

Short name:

BlockSilent

Tagline:

"Automatically manage your phone sound mode based on your location."

===== 3. MAIN CONCEPT  
=====

The application detects whether the user has entered or exited a predefined college location.

Example:

User enters CSE Block ↓ Geofence detects entry ↓ Application checks assigned rule ↓ Silent Mode activated ↓ Notification displayed

When the user leaves:

User exits CSE Block ↓ Geofence detects exit ↓ Previous sound mode restored ↓ Notification displayed

The system should support different rules for different locations.

===== 4. MAIN FEATURES  
=====

Implement the following features.

## FEATURE 1: LOCATION-BASED SILENT MODE

The user can create a location.

For each location store:

- Location name
- Latitude
- Longitude
- Radius
- Sound mode
- Enabled/disabled status

Sound modes:

- Silent
- Vibrate
- Normal/Ring

Example:

CSE Block Latitude: automatically obtained from map/current location Longitude: automatically obtained Radius: 50 meters Mode: Silent Status: Enabled

When the user enters the location: Automatically activate the configured sound mode.

When the user exits: Restore the previous sound mode.

===== 5. MULTIPLE BLOCK SUPPORT  
=====

Allow users to create multiple locations.

Example:

1. CSE Block → Silent
2. Library → Silent
3. Computer Lab → Vibrate
4. Seminar Hall → Silent
5. Canteen → Normal
6. Auditorium → Silent
7. Examination Hall → Silent

Display all saved locations in a clean list.

Each location should have:

- Name
- Current rule
- Radius
- Status

- Edit button
- Delete button
- Enable/Disable switch

## ===== 6. MAP LOCATION SELECTION =====

Create a map screen.

The user should be able to:

1. Open map.
2. View current location.
3. Search/select a location.
4. Tap on the map.
5. Set latitude and longitude.
6. Select radius.
7. Enter location name.
8. Select sound mode.
9. Save the geofence.

Display the selected location as a marker.

Display the geofence radius as a visible circle.

Example:

CSE Block  (50 meter circle)

Provide radius options:

25 meters 50 meters 75 meters 100 meters Custom radius

The user should be able to modify the radius later.

## ===== 7. CURRENT LOCATION DETECTION =====

Create a Current Location section.

Display:

Current Location: CSE Block

Current Sound Mode: Silent

Automation: ACTIVE

Location Permission: Granted

GPS: ON

If GPS is disabled:

Show:

"Location services are disabled. Please enable location services to use automatic block detection."

If permission is denied:

Show an appropriate permission message and button to open Android settings.

===== 8. TIMETABLE FEATURE  
=====

Create a timetable module.

The user should be able to add:

- Subject name
- Day
- Start time
- End time
- Block/location
- Sound mode

Example:

Monday 9:00 AM – 10:00 AM Mobile App Development CSE Block Silent

Tuesday 10:00 AM – 11:00 AM Computer Lab Computer Lab Vibrate

Display timetable in a weekly format.

Allow:

- Add
- Edit
- Delete
- Enable/disable timetable entries

The system can use both location and timetable information.

===== 9. SMART RULE ENGINE  
=====

Create a simple rule engine.

The final sound mode should be decided using:

Location + Day + Time + Rule

Example:

IF Current location = CSE Block AND Current time = 9:00 AM–10:00 AM AND Rule = Silent

THEN:

Activate Silent Mode.

Another example:

IF Current location = Lab AND Rule = Vibrate

THEN:

Activate Vibrate Mode.

If no rule matches: Do not automatically change the sound mode.

===== 10. SOUND MODE MANAGEMENT  
=====

Implement Android sound mode control using supported Android APIs and permissions.

Modes:

NORMAL VIBRATE SILENT

Before changing the sound mode:

Store the previous sound mode.

Example:

Previous mode = Vibrate

User enters CSE Block

New mode = Silent

User leaves CSE Block

Restore = Vibrate

Do NOT always restore to Normal/Ring.

The application must attempt to restore the user's previous mode.

If Android version or device restrictions prevent direct automatic changing of sound mode, clearly handle this using the appropriate Android notification policy / Do Not Disturb permission flow and guide the user to enable the required permission.

#### ===== 11. IMPORTANT CONTACT / EXCEPTION FEATURE =====

Create an Important Contacts section.

Allow the user to mark selected contacts as important.

Example:

Parents Guardian Emergency Contact

Provide an option:

“Allow important calls during silent mode”

Use Android-supported notification/DND mechanisms where possible.

Do not claim that the application can bypass Android restrictions if the operating system does not allow it.

#### ===== 12. EMERGENCY OVERRIDE =====

Create an Emergency Override feature.

Add a clearly visible button:

EMERGENCY OVERRIDE

When pressed:

Temporarily disable automatic sound-mode changes.

Options:

15 minutes 30 minutes 1 hour Until manually enabled

During override:

Automation: TEMPORARILY DISABLED

After the selected duration: Automatically enable automation again.

Also provide:

Enable Automation Now

===== 13. NOTIFICATION SYSTEM  
=====

Create Android notifications.

When entering a block:

"Silent Mode Activated" "You entered CSE Block."

When leaving:

"Sound Mode Restored" "You left CSE Block."

When entering a vibrate location:

"Vibrate Mode Activated" "You entered Computer Lab."

When automation is disabled:

"BlockSilent Automation Disabled"

When override expires:

"BlockSilent Automation Enabled"

Notifications should use appropriate Android notification channels.

Allow the user to enable/disable notifications from settings.

===== 14. ACTIVITY HISTORY  
=====

Create an Activity History screen.

Store:

- Date
- Time
- Location
- Event type
- Previous mode
- New mode

Example:

10 Sep 2026 09:02 AM Entered CSE Block Previous: Normal Changed to: Silent

10 Sep 2026 10:05 AM Exited CSE Block Previous: Silent Restored: Normal

Display newest activity first.

Add:

Clear History

option.

===== 15. HOME DASHBOARD  
=====

Create an attractive dashboard.

Home screen should display:

---

BlockSilent Location-Based Block Assistant

---

Current Location: CSE Block

Current Mode:  Silent

Automation: ☒ Active

GPS: ☒ Enabled

Active Rule: CSE Block → Silent

---

Quick Actions:

[ Add Block ]

[ View Blocks ]

[ Timetable ]

[ Activity History ]

[ Emergency Override ]

---

Show today's activity summary.

Example:

Blocks configured: 5 Automation: Active Today's location changes: 4

===== 16. MY BLOCKS SCREEN  
=====

Create a screen:

"My Blocks"

Display cards.

Example:

---

 CSE Block

Mode: Silent Radius: 50 m Status: Active

**[Edit] [Disable]**

---

 Library

Mode: Silent Radius: 40 m Status: Active

**[Edit] [Disable]**

Include:

- Add New Block

===== 17. ADD BLOCK SCREEN  
=====

Fields:

Block/Location Name

Latitude

Longitude

Radius

Sound Mode

Enable Automation

Buttons:

[Use Current Location]

[Select on Map]

[Save Block]

Validation:

- Name cannot be empty.
- Latitude must be valid.
- Longitude must be valid.
- Radius must be greater than zero.
- Sound mode must be selected.

===== 18. SETTINGS SCREEN  
=====

Create:

Settings

Sections:

GENERAL

- Enable Automation
- Notifications
- Default Sound Mode

LOCATION

- Location Permission
- GPS Status
- Geofence Radius

SOUND

- Normal
- Vibrate
- Silent

SAFETY

- Important Contacts
- Emergency Override

PRIVACY

- Location Usage Information

- Clear Activity History
- Delete All Local Data

## ABOUT

- App Name
- Version
- Developer Information

## ===== 19. PRIVACY FEATURE =====

Because this application uses location data, include a privacy section.

Clearly explain:

“The application uses location information only to detect whether the user has entered or exited configured college areas. Location data should be processed locally whenever possible and should not be unnecessarily shared with external servers.”

Do not collect unnecessary personal information.

Provide:

Clear Local Data

This should delete:

- Saved blocks
- Timetable
- History
- Settings

Ask for confirmation before deletion.

## ===== 20. BATTERY OPTIMIZATION =====

The application should avoid unnecessary continuous GPS tracking.

Prefer Android Geofencing APIs for location-triggered events.

Do not continuously request high-frequency GPS updates unless required.

Display battery optimization guidance if required.

Add a settings/help section:

“Battery Optimization”

Explain that users may need to exclude the application from battery optimization on some devices for reliable background geofence events.

Handle manufacturer-specific restrictions gracefully.

## ===== 21. PERMISSION MANAGEMENT =====

Request only necessary permissions.

Depending on Android version, handle:

Location permission Background location permission where required Notification permission where required Notification policy / Do Not Disturb access where required for sound-mode control

Do not request all permissions immediately.

Use a proper permission flow:

Explain permission ↓ Request permission ↓ Check result ↓ If denied → explain ↓ Allow user to open settings

Do not crash when permissions are denied.

## ===== 22. FIRST-TIME SETUP =====

When the application is opened for the first time:

Show onboarding screens.

Screen 1: Welcome to BlockSilent

Screen 2: How location detection works

Screen 3: Why location permission is required

Screen 4: How automatic sound mode works

Screen 5: Battery and privacy information

Then:

[Get Started]

After this, show permission setup.

## ===== 23. APP NAVIGATION =====

Use a simple bottom navigation or navigation structure.

Recommended:

Home Blocks Timetable History Settings

Floating action button:

- Add Block

===== 24. UI DESIGN  
=====

Create a modern, colorful but professional college-project UI.

Requirements:

- Clean dashboard
- Rounded cards
- Good spacing
- Modern icons
- Clear typography
- Light theme
- Optional dark theme if practical
- Responsive layouts
- Accessibility-friendly text sizes
- No overcrowded screens

Use different visual indicators for:

Active Inactive Silent Vibrate Normal GPS enabled GPS disabled Permission denied

Do not use excessive animations.

The application must remain easy to understand for students.

===== 25. DATABASE DESIGN  
=====

Use Room Database or SQLite.

Create tables/entities such as:

BLOCKS

id name latitude longitude radius soundMode enabled

TIMETABLE

id subjectName day startTime endTime blockId soundMode enabled

## HISTORY

id timestamp blockName eventType previousMode newMode

## IMPORTANT\_CONTACTS

id name phoneNumber enabled

## SETTINGS

id automationEnabled notificationsEnabled defaultSoundMode overrideUntil

Use proper relationships where required.

===== 26. GEOFENCING LOGIC  
=====

Implement geofencing correctly.

For every enabled block:

Create geofence using:

latitude longitude radius

Monitor:

GEOFENCE\_ENTER GEOFENCE\_EXIT

On ENTER:

1. Identify block.
2. Check whether automation is enabled.
3. Check emergency override.
4. Check timetable/rules if enabled.
5. Save current sound mode.
6. Change to configured mode.
7. Store history.
8. Show notification.

On EXIT:

1. Identify block.
2. Check automation.
3. Check whether the block's rule is active.
4. Restore the previously stored sound mode.
5. Store history.
6. Show notification.

Prevent duplicate actions caused by repeated geofence events.

## ===== 27. MULTIPLE GEOFENCE CONFLICT HANDLING =====

Handle situations where two geofences overlap.

Example:

Library radius = 50m CSE Block radius = 50m

If both are detected:

Use a priority system.

Priority:

1. Examination Hall
2. Classroom
3. Laboratory
4. Library
5. Other configured locations

Allow the user to change priority if possible.

Do not repeatedly switch between modes when overlapping geofences trigger.

## ===== 28. OFFLINE FUNCTIONALITY =====

The core application should work without internet after setup wherever Android allows it.

Saved blocks, timetable, rules and history should remain available locally.

Google Maps may require internet for map tiles/search, but geofence configuration should use saved coordinates.

## ===== 29. ERROR HANDLING =====

Handle:

GPS disabled Location permission denied Background location denied Notification permission denied  
DND permission unavailable Google Play Services unavailable Database errors Invalid coordinates  
Invalid radius Duplicate blocks Geofence registration failure Device reboot Application restart

Show user-friendly messages.

Never display raw technical exceptions to the user.

## ===== 30. DEVICE RESTART HANDLING =====

After device reboot:

Restore saved geofences where Android allows it.

Check:

- Automation enabled
- Saved blocks
- Required permissions
- Current override state

Do not lose saved data.

===== 31. SECURITY  
=====

Do not store unnecessary sensitive information.

Validate all user inputs.

Keep local database protected using standard Android storage practices.

Do not expose location data unnecessarily.

===== 32. ACCESSIBILITY  
=====

Support:

- Readable font sizes
- Proper contrast
- Content descriptions for icons
- Buttons large enough to tap
- Clear error messages
- No color-only status indicators

===== 33. PROJECT STRUCTURE  
=====

Create a clean Android project structure.

Suggested structure:

```
app/ |—— src/main/java/ | |—— activities/ | |—— fragments/ | |—— adapters/ | |——  
models/ | |—— database/ | |—— geofence/ | |—— notifications/ | |—— services/ | |——  
utils/ | |—— repositories/ | |—— src/main/res/ | |—— layout/ | |—— drawable/ | |——  
mipmap/ | |—— values/ | |—— menu/
```

Use meaningful class names.

===== 34. RECOMMENDED CLASSES  
=====

Create appropriate classes such as:

MainActivity HomeFragment BlocksFragment TimetableFragment HistoryFragment SettingsFragment

AddBlockActivity EditBlockActivity MapActivity

BlockAdapter TimetableAdapter HistoryAdapter

BlockEntity TimetableEntity HistoryEntity ContactEntity SettingsEntity

AppDatabase BlockDao TimetableDao HistoryDao

GeofenceManager GeofenceBroadcastReceiver SoundModeManager NotificationHelper  
PermissionManager LocationHelper RuleEngine

Use appropriate architecture and avoid putting all logic inside MainActivity.

===== 35. TESTING REQUIREMENTS  
=====

Test the application for:

1. Entering a block
2. Leaving a block
3. Silent mode activation
4. Vibrate activation
5. Previous mode restoration
6. Multiple blocks
7. Timetable rules
8. Emergency override
9. Notifications
10. GPS disabled
11. Permission denied
12. Device restart
13. App restart
14. Invalid block data
15. Overlapping geofences

Add logs for debugging but do not expose technical logs to normal users.

===== 36. DEMO MODE  
=====

Create an optional DEMO/TEST mode for college project demonstration.

This is important because actual GPS movement may be difficult during a classroom presentation.

Demo mode should allow the developer/tester to simulate:

ENTER CSE BLOCK

EXIT CSE BLOCK

ENTER LIBRARY

ENTER LAB

EXIT LOCATION

The demo mode should trigger the same application logic as the real geofence events.

Clearly label it:

DEMO MODE

Do not allow demo mode to accidentally affect real user settings unless intentionally selected.

===== 37. PROJECT DASHBOARD STATUS  
=====

Home screen should show:

Location Permission: Granted / Not Granted

GPS: ON / OFF

Automation: ACTIVE / INACTIVE

Current Block: CSE Block / Outside Configured Area

Sound Mode: Silent / Vibrate / Normal

Emergency Override: Active / Inactive

===== 38. INITIAL SAMPLE DATA  
=====

For testing, provide sample blocks:

CSE Block Mode: Silent Radius: 50m

Library Mode: Silent Radius: 40m

Computer Lab Mode: Vibrate Radius: 50m

Canteen Mode: Normal Radius: 50m

Auditorium Mode: Silent Radius: 60m

Allow users to edit or delete these sample entries.

===== 39. IMPORTANT ANDROID COMPATIBILITY  
=====

The application must follow current Android permission and background-execution restrictions.

Do not use deprecated APIs unnecessarily.

Before implementing any Android API related to changing sound mode, verify what Android allows on the selected target SDK.

If direct silent-mode switching is restricted:

1. Explain why.
2. Request the appropriate Notification Policy Access where supported.
3. Guide the user through Android Settings.
4. Gracefully handle unsupported devices.
5. Never falsely report that silent mode was changed if the operation failed.

===== 40. DEVELOPMENT PROCESS  
=====

Do not generate everything blindly in one step.

First:

1. Analyze requirements.
2. Create project structure.
3. Set up Gradle dependencies.
4. Create database.
5. Create UI.
6. Implement navigation.
7. Implement block management.
8. Implement map.
9. Implement geofencing.
10. Implement sound-mode manager.
11. Implement notifications.
12. Implement timetable.
13. Implement history.
14. Implement emergency override.
15. Implement settings.
16. Implement permissions.
17. Implement demo mode.
18. Test all features.
19. Fix compilation errors.
20. Fix runtime errors.
21. Build the final APK.

After every major implementation step:

- Check compilation.
- Fix errors.
- Check imports.
- Check AndroidManifest.
- Check Gradle configuration.
- Check runtime permissions.
- Check database migrations if required.

===== 41. FINAL REQUIREMENT  
=====

The final application should be:

Functional Stable Beginner-friendly Colorful Modern Privacy-aware Battery-conscious Location-based  
Easy to demonstrate Suitable for a college Mobile Application Development project.

The final workflow should be:

USER ↓ Open BlockSilent ↓ Grant required permissions ↓ Add/select college blocks ↓ Set radius and  
sound mode ↓ Register geofences ↓ Move into configured block ↓ Geofence detects ENTER ↓ Rule  
Engine checks location/time ↓ SoundModeManager changes phone mode ↓ Notification shown ↓  
Activity stored ↓ User leaves block ↓ Geofence detects EXIT ↓ Previous sound mode restored ↓  
Notification shown ↓ Activity stored

===== 42. WHAT I EXPECT FROM YOU  
=====

Build the project completely.

Do not only provide explanations.

Create the actual Android project files, Java classes, XML layouts, Gradle configuration, AndroidManifest  
permissions, database classes, adapters, resources and required components.

Make sure all code is connected and runnable.

Whenever you create a file, clearly tell me:

- File name
- File path
- Purpose

If a dependency or API has changed in the latest Android version, use the currently supported  
approach.

If a feature cannot be implemented exactly because of Android OS restrictions, do not fake it.  
Implement the closest supported solution and explain the limitation.

At the end:

1. Build the application.
2. Fix all compilation errors.
3. Fix all runtime errors you can identify.
4. Provide the exact steps to run it in Android Studio.
5. Provide the required permissions.
6. Explain how to test geofencing.
7. Explain how to test Demo Mode.
8. Explain how to generate the APK.
9. Provide a final feature checklist.

Start by creating the project architecture and implementation plan, then proceed step-by-step with the actual implementation.